

The Effect of LoRA Initialization on Parameter-Efficient GPT-2 Fine-Tuning

İlker Yayalar, Hakan Boğan

Department of Computer Engineering, İzmir Institute of Technology (İYTE)

Abstract

We investigate how LoRA initialization influences training behavior and downstream performance in parameter-efficient fine-tuning (PEFT). We compare three initialization strategies: default, weight-distribution, and SVD-based initialization. On Shakespearean sonnet generation, weight-distribution initialization achieves the strongest LoRA result, reaching a chrF score of 40.09 with only 0.44M trainable parameters (0.36% of the model) and retaining 97.9% of the chrF performance of full fine-tuning while updating $281\times$ fewer parameters. In contrast, the three initialization strategies finish within 0.1 percentage point of one another on Quora paraphrase detection, suggesting that initialization has a stronger effect in low-resource generation than in large-scale classification. To provide a controlled experimental backbone, we reimplement GPT-2 and AdamW from scratch, transfer pretrained GPT-2 weights into our implementation, and validate both components against reference implementations. Overall, the results demonstrate that statistically informed LoRA initialization can improve parameter-efficient adaptation under data-constrained settings while approaching full fine-tuning performance with substantially fewer trainable parameters.

Keywords: LoRA, Parameter-Efficient Fine-Tuning, GPT-2, Initialization Strategies, AdamW, Low-Resource Generation

1. Introduction

Parameter-efficient fine-tuning (PEFT), particularly LoRA, offers an efficient alternative to full model fine-tuning by adapting large language models with only a small fraction of trainable parameters. In this work, we investigate different LoRA initialization strategies with a focus on improving convergence and maintaining downstream performance. As the experimental backbone, we implement GPT-2 and the AdamW optimizer from scratch based on the Stanford CS224N framework and validate them against reference implementations. Our results show that weight-distribution initialization retains 97.9% of full fine-tuning chrF on sonnet generation using only 0.36% of the trainable parameters, while LoRA variants recover about 94% of full fine-tuning accuracy on paraphrase detection.

2. Background

Decoder-only Transformer: GPT-2 stacks identical Transformer blocks [11] that process tokens left to right. Each block applies masked (causal) multi-head self-attention followed by a position-wise feed-forward network, with residual connections around both sublayers. GPT-2 places layer normalization before each sublayer (pre-LN), which stabilizes gradient flow and removes the need for a learning-rate warmup that the original post-LN design required [1]. The output projection reuses (ties) the input token-embedding matrix.

AdamW: Adam maintains exponential moving averages of the gradient and its square and applies bias correction [4]; AdamW decouples weight decay from the gradient update, applying it as a direct multiplicative shrinkage of the parameters [5].

Parameter-efficient fine-tuning: LoRA freezes the pretrained weights and learns a low-rank additive update $\Delta W = (\alpha/r)$ BA for selected projections, drastically reducing the number of trainable and stored parameters [3].

Generation metrics: chrF is a character n-gram F-score suited to morphology- and form-sensitive text [7]; BLEU is a word n-gram precision metric with a brevity penalty [6]. We treat chrF as primary and BLEU as complementary, following our task definition.

3. Methodology

Our methodology consists of four main components: a from-scratch GPT-2 backbone, a custom AdamW optimizer, downstream task adaptations, and a LoRA-based PEFT extension with different initialization strategies. Figure 1 illustrates the implemented pre-LayerNorm GPT-2 block, the LoRA-augmented attention projections, and the weight-tied language-modeling head. We describe each component.

3.1 GPT-2 backbone

Masked multi-head self-attention: For each head we

compute scaled dot-product attention

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M_{\text{causal}} + M_{\text{pad}}\right)V \quad (1)$$

with head dimension $d_k = 64$. The causal mask M_{causal} is built from a lower-triangular matrix (`torch.tril`); positions corresponding to future tokens are set to -10^4 before the softmax so their post-SoftMax weight is approximately zero. The padding mask M_{pad} , derived from the attention mask, is added in the same additive form, so variable-length sequences batch cleanly. We use the finite value -10^4 rather than $-\infty$ for numerical stability.

Pre-LayerNorm block: Each block applies normalization before the sublayer and adds the residual afterward:

$$\tilde{h} = h + \text{Drop}\left(W_o \text{Attn}(\text{LN}_1(h))\right) \quad (2)$$

$$h' = \tilde{h} + \text{Drop}\left(W_2 \text{GELU}\left(W_1 \text{LN}_2(\tilde{h})\right)\right) \quad (3)$$

The dense-projection-plus-dropout-plus-residual step deliberately does not re-apply normalization; the GELU activation [2] follows GPT-2.

Embeddings and weight tying: Input representations are the sum of learned token and positional embeddings followed by dropout. Output logits reuse the transposed token-embedding matrix, logits $= h E^T$ with $E \in \mathbb{R}^{V \times d}$, which avoids a separate $V \times d$ output matrix; for $V = 50,257$ and $d = 768$ these ties $\approx 38.6M$ parameters.

Weight transfer: We load the official GPT-2 weights into our modules [12]. The principal remapping concerns attention: HuggingFace stores Q, K, V as one fused 1D-convolution weight, which we slice into three separate linear projections and transpose to match the linear-layer convention.

3.2 AdamW optimizer

Our step uses the efficient bias-correction form. With first and second moments

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (4)$$

we fold both bias corrections into a single step size

$$\alpha_t = \alpha \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t}, \quad \theta_t = \theta_{t-1} - \alpha_t \frac{m_t}{\sqrt{v_t} + \epsilon} \quad (5)$$

and then apply decoupled weight decay as a multiplicative shrinkage $\theta_i \leftarrow \theta_i (1 - \alpha\lambda)$ after the gradient step [5].

3.3 Task heads

Sentiment: The last non-padding token representation is passed through dropout and a linear classifier to the label space. Two regimes are compared: last-linear-layer freezes the backbone and trains only the head, while full-model trains all parameters.

Paraphrase Detection: We use the last-token representation with a two-way linear head trained with cross-entropy against the binary duplicate label. We note this is a design choice: the

assignment also describes a cloze-style formulation that generates the words “yes”/“no”. A practical consequence of our setup is that the train and test prompt templates differ, which we revisit in the analysis.

Sonnet generation: We attach the weight-tied head to the full final hidden-state sequence and train with shifted cross-entropy (language-modeling) loss on the training sonnets. Generation conditions on the first three lines of a held-out sonnet and continues autoregressively with nucleus (top-p) sampling at $p = 0.9$ and temperature 1.2.

3.4 LoRA extension

For PEFT we add a low-rank update to each attention projection,

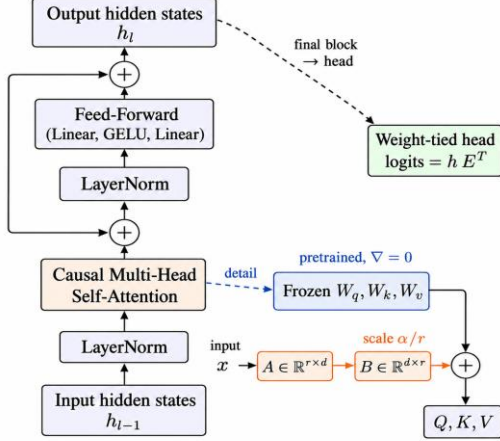


Figure 1: One pre-LayerNorm GPT-2 block (left, bottom-to-top), the LoRA-augmented attention projection (right): each frozen weight W is summed with a low-rank path $(\alpha/r)BA$ that is the only trainable component in LoRA mode; and the weight-tied output head used for language modeling.

$$W'x = Wx + \frac{\alpha}{r}B(Ax), \quad A \in \mathbb{R}^{r \times d}, \quad B \in \mathbb{R}^{d \times r} \quad (6)$$

where W is frozen and only the low-rank factors A and B are trained. We place adapters on the query, key, and value projections of all 12 layers (36 adapter pairs) with rank $r = 8$ and $\alpha = 16$ (scaling 2).

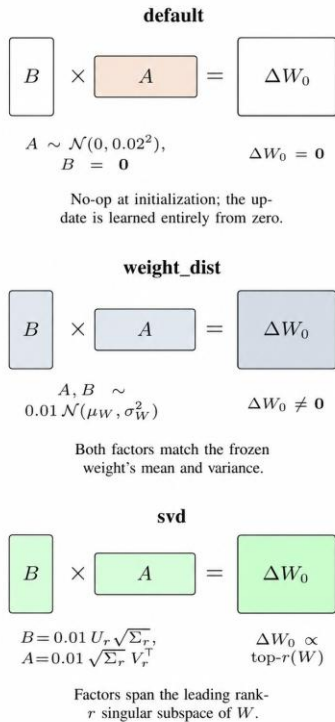


Figure 2: The three LoRA initialization schemes at the start of training. All use the same low-rank form $\Delta W = (\alpha/r)BA$ on each frozen Query/Key/Value projection ($r=8, \alpha=16$); they differ only in how the down-projection $A \in \mathbb{R}^{r \times d}$ (wide) and the up-projection $B \in \mathbb{R}^{d \times r}$ (tall) are

initialized. default starts as a no-op ($B = 0$); weight_dist starts from a small Gaussian update matched to the frozen weight’s statistics; svd starts aligned with the frozen weight’s dominant singular directions. Here μ_W, σ_W are the mean and standard deviation of the pretrained projection W , and U_r, Σ_r, V_r its truncated rank- r singular value decomposition.

We compare three initialization schemes for the LoRA factors A and B , as illustrated in Figure 2:

- **Default initialization:** A is sampled from $\mathcal{N}(0, 0.02^2)$, while B is initialized to zero. This produces an initial update of $\Delta W_0 = BA = 0$, meaning that the adapter begins without modifying the pretrained weights.

$$A \sim \mathcal{N}(0, 0.02^2), \quad B = 0$$

- **Weight-distribution initialization:** Both factors are sampled from a Gaussian distribution matched to the mean and standard deviation of the corresponding pretrained weight matrix and scaled by 0.01. This produces a small non-zero update within the numerical range of WW . This is the primary initialization strategy investigated in this study.

$$A, B \sim \mathcal{N}(\mu_W, \sigma_W^2), \text{ followed by}$$

$$A \leftarrow 0.01 A, B \leftarrow 0.01 B \quad \text{Hence, } \Delta W_0 = BA \neq 0,$$

- **SVD (Singular Value Decomposition of W) initialization:** A and B are initialized from a truncated rank- r singular value decomposition of the pretrained weight matrix W and scaled by 0.01. This aligns the initial update with the dominant singular directions of W .

$$W \approx U_r \Sigma_r V_r^T$$

and the LoRA factors are initialized from the truncated rank- r decomposition and scaled by 0.01.

We expect the weight-distribution initialization to provide a more effective starting point for LoRA optimization by introducing a small pretrained-weight-aware update from the beginning of training. Compared with the zero-update default initialization, this may improve early optimization and convergence, while remaining less structurally constrained than SVD initialization.

3.5 Correctness validation

Before any training we confirm that our backbone reproduces the HuggingFace GPT-2 last hidden state to an absolute tolerance of 10^{-1} , and that our optimizer step matches a provided reference tensor to 10^{-6} . These checks isolate task results from implementation error.

4. Experiments

4.1 Datasets and statistics

Table 1 reports statistics computed directly from the provided splits, with sequence lengths measured in GPT-2 byte-pair tokens. The four tasks span very different input regimes: SST [10] sentences are short (≈ 22 tokens), CFIMDB reviews are an order of magnitude longer (≈ 218 tokens, up to the model context), Quora supplies a large but class-imbalanced corpus (63%/37%), and the sonnet corpus is tiny (131 training poems).

Dataset	Task	Cls.	Train	Dev	Test	Avg. len	Label balance
SST	5-way sentiment	5	8,544	1,101	2,210	22.4 tok	-
CFIMDB	binary sentiment	2	1,707	245	488	217.6 tok	$\approx 50/50$
Quora	paraphrase detection	2	283,011	40,429	80,860	27.0 tok	63/37 %
Sonnets	poem generation	-	131	12	12	160.0 tok	-

Table 1: Dataset statistics, computed from the provided files.

Average length is in GPT-2 tokens: per sentence (SST, CFIMDB), per concatenated question pair (Quora), and per poem (Sonnets). SST training-class counts are {1092, 2218, 1624, 2322, 1288} for the five ordinal classes; the neutral class is the minority among the central labels. The sonnet corpus contains

131 training poems (≈ 14 lines each) plus 12 held-out development and 12 held-out test poems

The backbone is GPT-2 small: 12 layers, hidden size 768, 12 heads, context length 1024, and a vocabulary of 50,257, totaling 124.44M parameters with the tied output head. The three adaptation regimes we compare differ sharply in how many of these parameters they train; we report each regime’s parameter budget next to its generation quality in Table 6.

4.2 Setup

All experiments use the GPT-2 small backbone with the AdamW optimizer. Cloud GPU resources were used for some experiments only to accelerate training and reduce experimentation time. However, the system was designed to run efficiently on consumer-grade hardware and can be executed effectively on a laptop equipped with an Nvidia RTX 4060 GPU.

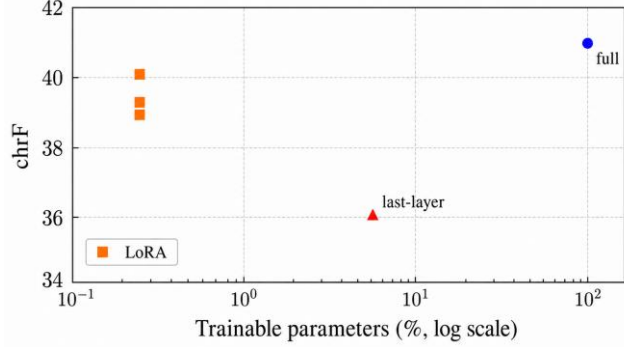


Figure 3: Quality versus trainable-parameter cost. The three LoRA variants (orange) reach near full fine-tuning chrF (blue) at 0.36% trainable parameters, dominating last-layer fine-tuning (red).

On full fine-tuning value; the frozen-backbone probe uses the released default of 10^{-3} . Batch size is 64 (SST) and 8 (CFIMDB) with head dropout 0.3. LoRA runs use $r=8$, $\alpha=16$ on the Q/K/V projections of all 12 layers.

Task	LR	Ep.	Batch	Drop.	Decode
Sentiment	10^{-5}	10	64/8	0.3	-
Paraphrase	$4e-5$	10	128	-	-
Sonnet	$1e-5$	10	8	-	$p=0.9$, $T=1.2$

Table 2: Experimental training configurations for the downstream task

4.3 Evaluation Metrics

We evaluate the downstream tasks using metrics appropriate to their respective objectives. For the sentiment classification and paraphrase detection tasks, we report **accuracy** and **macro-averaged F1 score (Macro-F1)**. Accuracy measures the proportion of correctly classified examples,

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}}, \quad (7)$$

whereas Macro-F1 computes the F1 score independently for each class and averages the resulting values,

$$\text{Macro}_{F1} = \frac{1}{C} \sum_{c=1}^C F1_c. \quad (8)$$

Macro-F1 is included because it gives equal importance to each class and is therefore useful for evaluating performance under class imbalance, particularly for the Quora paraphrase dataset. Class-wise recall is additionally reported as a diagnostic measure to examine whether performance is distributed consistently across labels. The paper’s sentiment and paraphrase experiments already use accuracy, Macro-F1, and class-level recall in this manner.

For sonnet generation, we use **chrF** as the primary evaluation metric and **BLEU** as a complementary metric. chrF evaluates the similarity between generated and reference texts using character-level nn-gram precision and recall, making it suitable for

capturing morphological and surface-form similarities. BLEU instead measures word-level nn-gram precision while incorporating a brevity penalty. Because generated sonnets may preserve lexical and morphological patterns even when exact word sequences differ from the reference, chrF is treated as the primary generation metric in this study. The same metric choice is maintained in the reported sonnet experiments.

In addition to task performance, we evaluate **parameter efficiency** by reporting the number and percentage of trainable parameters for each adaptation regime. This allows full fine-tuning, last-layer fine-tuning, and LoRA-based adaptation to be compared not only in terms of predictive or generation quality, but also in terms of optimization cost. For LoRA, the trainable-parameter ratio is computed relative to the 124.44M-parameter GPT-2 backbone,

$$\text{TR} = \text{Trainable Ratio}(\%) = \frac{N_{\text{trainable}}}{N_{\text{total}}} \times 100. \quad (9)$$

Adapter storage is additionally derived from the number of trainable parameters assuming 32-bit floating-point parameters.

5. Results

5.1 Sentiment Classification

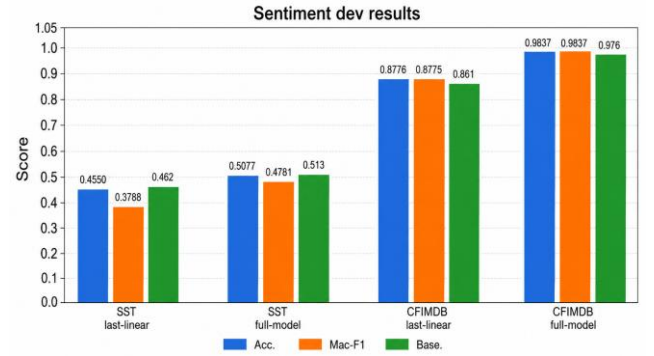


Figure 4: SST per-class dev recall. The neutral class is hardest for both regimes (full-model recall 0.183, last-linear 0.079); the frozen-backbone probe collapses on the rare extreme classes.

Full-model fine-tuning consistently provided stronger sentiment classification performance than training only the final linear layer. On SST, full-model fine-tuning achieved an accuracy of 0.5077 and a Macro-F1 score of 0.4781, compared with 0.4550 accuracy and 0.3788 Macro-F1 for the last-linear setting. The improvement was more pronounced on CFIMDB, where full-model fine-tuning reached 0.9837 for both accuracy and Macro-F1, whereas the last-linear configuration achieved 0.8776 accuracy and 0.8775 Macro-F1. These results indicate that updating the complete pretrained representation is particularly beneficial for the binary CFIMDB task, while SST remains substantially more challenging because it requires finer-grained discrimination across five sentiment classes.

Data	Regime	Acc.	Mac-F1	Base.
SST	last-linear	0.4550	0.3788	0.462
	full-model	0.5077	0.4781	0.513
CFIMDB	last-linear	0.8776	0.8775	0.861
	full-model	0.9837	0.9837	0.976

Table 3: results of sentiment classification

The class-level results further reveal that SST performance is not uniformly distributed across sentiment categories. The neutral class was the most difficult for both adaptation regimes, with recall values of 0.183 under full-model fine-tuning and 0.079 when only the final layer was trained. This suggests that examples near the center of the sentiment scale are more difficult to distinguish than strongly positive or negative examples. The frozen-backbone configuration also showed weaker behavior on the less frequent extreme classes, indicating that a fixed pretrained representation is less effective for capturing the finer distinctions required by the five-class task.

5.2 Paraphrase Detection and the LoRA ablation

Metric	Value	Support
Accuracy	0.8978	40,429
Macro-F1	0.8912	-
Recall (non-dup., 0)	0.906	25,537
Recall (duplicate, 1)	0.884	14,892

Table 4: results of paraphrase detection

The full paraphrase classifier achieved strong performance on the Quora development set, reaching 0.8978 accuracy and 0.8912 Macro-F1. Performance was also relatively balanced across the two classes: recall was 0.906 for non-duplicate question pairs and 0.884 for duplicate pairs. This is notable given the approximately 63/37 class distribution and indicates that the model does not obtain its overall accuracy simply by favoring the majority class.

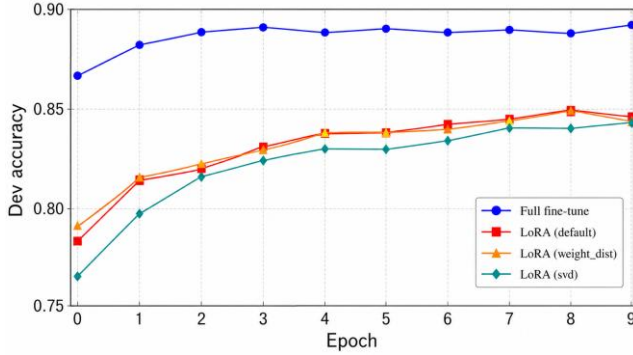


Figure 5: Quora development accuracy per epoch for the four paraphrase runs. The three LoRA initializations are nearly indistinguishable and settle about five accuracy points below full fine-tuning, in contrast to the sonnet ablation where the initializations separated.

A controlled comparison between full fine-tuning and the three LoRA initialization strategies produced a different pattern. Full fine-tuning reached 0.896 accuracy, whereas all LoRA variants converged to approximately 0.844–0.845 final accuracy. Their peak development accuracies were also close, ranging from 0.844 to 0.849. Weight-distribution initialization achieved the highest observed peak at 0.849, but the difference from the default initialization was only 0.001. Consequently, there is no substantial performance separation between the three initialization strategies on this task.

Regime	Trainable	% TR.	Dev acc	Peak
Full fine-tune	124.44 M	100.00	0.896	0.896
LoRA (default)	0.44 M	0.36	0.845	0.848
LoRA (weight_dist)	0.44 M	0.36	0.844	0.849
LoRA (svd)	0.44 M	0.36	0.844	0.844

Table 5: Compare between full fine-tune and LoRA distributions-1

Despite this reduction relative to full fine-tuning, each LoRA configuration recovered approximately 94% of full fine-tuning accuracy while updating only 0.36% of the model parameters. The similar learning trajectories of default, weight-distribution, and SVD initialization suggest that, under this large-scale classification setting, the choice of LoRA initialization has limited influence on the final solution

5.4 Sonnet Generation and the LoRA ablation

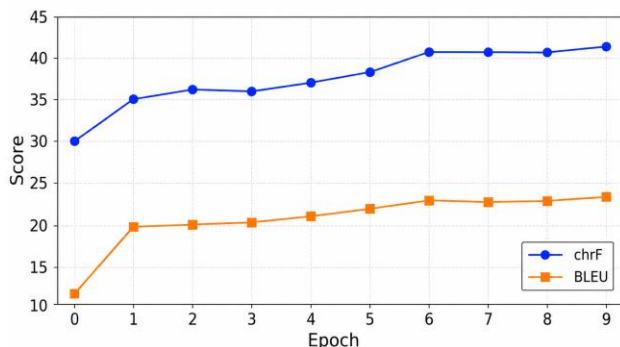


Figure 6: Full-model sonnet learning curve. chrF improves from 30.02 to 41.44 over ten epochs and plateaus after epoch six; BLEU tracks chrF at a lower absolute level

Sonnet generation exhibited a clearer sensitivity to the adaptation strategy. Full-model fine-tuning achieved a chrF score of 40.95 and a BLEU score of 23.75 on the held-out development poems. During training, chrF increased rapidly from 30.02 in the first recorded epoch and reached 40.76 by epoch six, after which improvements became relatively small. The highest per-epoch chrF score was 41.44, while the final checkpoint used for evaluation achieved 40.95, indicating that most of the learning occurred during the earlier stages of training.

Regime	Trainable	% tr.	chrF	BLEU
Full fine-tune	124.44 M	100.00	40.95	23.75
Last-layer	7.09 M	5.70	36.11	21.58
LoRA (default)	0.44 M	0.36	39.29	23.07
LoRA (weight_dist)	0.44 M	0.36	40.09	23.54
LoRA (svd)	0.44 M	0.36	39.13	23.10

Table 6: Compare between full fine-tune and LoRA distributions-2

Among the parameter-efficient approaches, weight-distribution initialization produced the strongest generation performance, reaching 40.09 chrF and 23.54 BLEU. Default LoRA achieved 39.29 chrF and 23.07 BLEU, while SVD initialization reached 39.13 chrF and 23.10 BLEU. In contrast to paraphrase detection, the initialization strategies therefore show a measurable separation on the low-resource generation task. Weight-distribution initialization retained approximately 97.9% of the chrF performance of full fine-tuning, compared with approximately 95.9% for default initialization and 95.6% for SVD initialization.

Last-layer fine-tuning performed substantially worse, reaching 36.11 chrF and 21.58 BLEU despite training considerably more parameters than LoRA. All three LoRA configurations therefore achieved higher generation quality than last-layer fine-tuning. These results suggest that distributing low-rank updates across the attention projections is more effective for this task than concentrating trainable capacity in the final layer, and that the starting point of those low-rank updates can influence performance when training data are limited.

5.5 Parameter Efficiency

The adaptation strategies differ substantially in the number of parameters that must be updated during training. Full fine-tuning optimizes all 124.44 million parameters of GPT-2, whereas last-layer fine-tuning updates 7.09 million parameters, corresponding to 5.70% of the model. Each LoRA configuration requires only approximately 0.44 million trainable parameters, or 0.36% of the complete model. LoRA therefore uses approximately 281 times fewer trainable parameters than full fine-tuning and approximately 16 times fewer than last-layer fine-tuning.

This reduction does not result in a proportional loss of generation quality. The best LoRA configuration retains 97.9% of full fine-tuning chrF while requiring an adapter of approximately 1.8 MB, compared with roughly 498 MB for the full model checkpoint under 32-bit parameter storage. Moreover, LoRA achieves higher sonnet-generation quality than last-layer fine-tuning despite using substantially fewer trainable parameters. The results therefore demonstrate that LoRA provides a more favorable trade-off between adaptation cost and downstream generation performance in the evaluated setting.

5.6 Qualitative Analysis

Generated sonnets generally preserved the conditioning prompt, adopted archaic vocabulary, and reproduced several structural characteristics of Shakespearean poetry. However, deviations from the expected 14-line structure, inconsistent meter, lexical artifacts, and semantic drift were occasionally observed.

6. Discussion and Conclusion

The results of this study show that the initialization of LoRA parameters can influence downstream performance, although the

magnitude of this effect depends strongly on the task. On Quora paraphrase detection, the default, weight-distribution, and SVD initializations converged to nearly identical accuracy values, indicating that different initialization strategies may become less important when sufficient training data are available. In contrast, the sonnet-generation experiments produced a clearer separation between initialization methods. Under the same number of training epochs and the same LoRA parameter budget, weight-distribution initialization achieved the strongest generation performance, reaching 40.09 chrF compared with 39.29 for the default initialization and 39.13 for SVD initialization. This result provides evidence that a pretrained-weight-aware statistical starting point can be beneficial in settings where the amount of downstream data is limited.

These findings suggest that parameter-efficient fine-tuning should not be considered only as a problem of reducing the number of trainable parameters. The initial distribution of those parameters can also affect how effectively the available optimization budget is used. A LoRA adapter initialized from statistical properties of the pretrained weights may begin optimization from a more favorable region of the parameter space than a generic initialization. This advantage was not evident in the large-scale paraphrase task, but it became observable in the low-resource sonnet-generation setting. Therefore, the effectiveness of an initialization strategy appears to be task- and data-dependent rather than universally superior.

The parameter-efficiency results further demonstrate the practical value of this approach. Full fine-tuning requires optimization of all 124.44 million GPT-2 parameters, whereas the evaluated LoRA configurations update only approximately 0.44 million parameters, corresponding to 0.36% of the model. Despite this substantial reduction, weight-distribution LoRA retained 97.9% of the chrF performance achieved by full fine-tuning. The resulting adapter requires approximately 1.8 MB of storage compared with about 498 MB for a full-model checkpoint. These characteristics make LoRA particularly attractive for experiments conducted with limited computational resources, smaller datasets, or systems with restricted GPU memory. However, the present experiments directly establish parameter and storage efficiency rather than an exact reduction in training time, VRAM consumption, or monetary cost; these aspects would require dedicated runtime and hardware measurements.

At the same time, the results also illustrate the current limitation of parameter-efficient adaptation. Although LoRA approaches the performance of full fine-tuning with a very small fraction of trainable parameters, it does not completely reproduce the performance of updating the entire model. Full fine-tuning remained the strongest configuration for sonnet generation, reaching 40.95 chrF and 23.75 BLEU, while the best LoRA configuration reached 40.09 chrF and 23.54 BLEU. Thus, PEFT methods provide a trade-off rather than a complete replacement for full-model optimization: they sacrifice a relatively small amount of performance in exchange for a substantial reduction in trainable parameters and storage requirements.

Overall, this study demonstrates that LoRA initialization is a meaningful component of parameter-efficient fine-tuning. Different initialization distributions may lead to nearly identical outcomes in some tasks, while in others a statistically informed initialization can produce better results within the same training budget. The weight-distribution strategy was particularly effective for low-resource sonnet generation, where it approached full fine-tuning performance while updating only a small subset of model parameters. These findings indicate that future PEFT research should consider not only adapter architecture and rank, but also the statistical properties of adapter initialization. With better initialization strategies, parameter-efficient methods may further reduce the gap between LoRA and full fine-tuning while enabling effective adaptation on smaller datasets and more resource-constrained hardware.

The conclusions should nevertheless be interpreted within the limitations of the experiments. Each configuration was evaluated with a single training run, development metrics were used

because test labels were unavailable, and the paraphrase setup contains a train-test prompt mismatch. Future work should therefore evaluate the proposed initialization across multiple random seeds, larger model variants, additional low-resource tasks, and direct measurements of GPU memory usage, training time, and computational cost.

7. References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. arXiv preprint arXiv:1607.06450, 2016.
- [2] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). arXiv preprint arXiv:1606.08415, 2016.
- [3] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In International Conference on Learning Representations, 2022.
- [4] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In International Conference on Learning Representations, 2015.
- [5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In International Conference on Learning Representations, 2019.
- [6] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, pages 311–318, 2002.
- [7] Maja Popović. chrF: Character n-gram F-score for automatic MT evaluation. In Proceedings of the Tenth Workshop on Statistical Machine Translation, pages 392–395. Association for Computational Linguistics, 2015.
- [8] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. Technical report, OpenAI, 2018.
- [9] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- [10] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, pages 1631–1642, 2013.
- [11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Advances in Neural Information Processing Systems, pages 5998–6008, 2017.
- [12] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pages 38–45, 2020.
- [13] F. Meng, Z. Wang, and M. Zhang, “PiSSA: Principal Singular Values and Singular Vectors Adaptation of Large Language Models,” 2024.
- [14] S. Wang, L. Yu, and J. Li, “LoRA-GA: Low-Rank Adaptation with Gradient Approximation,” 2024.
- [15] F. Paischer, L. Hauzenberger, T. Schmied, B. Alkin, M. P. Deisenroth, and S. Hochreiter, “One Initialization to Rule Them All: Fine-Tuning via Explained Variance Adaptation,” 2024.

Appendix

A-Per-Epoch Sonnet Metrics (Full Model)

To provide a detailed view of the sonnet-generation training process, the complete per-epoch training loss, chrF, and BLEU scores are reported below. The training loss decreases steadily throughout optimization, while both generation metrics improve substantially during the early epochs. chrF increases from 30.02 at the beginning of training to 41.44 at epoch 9, while BLEU increases from 11.89 to 23.47. Performance begins to stabilize after approximately the sixth epoch, indicating that most of the improvement occurs during the earlier stages of training. The final submitted checkpoint achieves 40.95 chrF and 23.75 BLEU.

Epoch	0	1	2	3	4	5	6	7	8	9	Final
Train loss	4.799	4.437	4.279	4.181	4.096	4.047	4.000	3.949	3.916	3.877	-
chrF	30.02	35.20	36.50	36.33	37.31	38.55	40.76	40.77	40.75	41.44	40.95
BLEU	11.89	19.95	20.19	20.45	21.11	22.03	23.10	22.97	23.05	23.47	23.75

Table 7: Full-model sonnet training: per-epoch loss and held-out chrF/BLEU. “Final” is the score of the last-epoch model used for submission.

B-Per-Epoch Paraphrase Development Accuracy

The complete development-accuracy trajectories for full fine-tuning and the three LoRA initialization strategies are shown below. Full fine-tuning remains consistently above all LoRA variants throughout training and reaches a final development accuracy of 0.896. The three LoRA configurations follow very similar trajectories. Default initialization reaches 0.845 final accuracy, weight-distribution initialization reaches 0.844, and SVD initialization also reaches 0.844. Their peak values remain close, supporting the observation that initialization has limited influence on the paraphrase task under this experimental configuration.

Epoch	0	1	2	3	4	5	6	7	8	9
Full fine-tune	0.867	0.882	0.890	0.893	0.891	0.893	0.892	0.893	0.892	0.896
LoRA (default)	0.786	0.815	0.820	0.830	0.837	0.838	0.842	0.844	0.848	0.845
LoRA (weight_dist)	0.789	0.816	0.822	0.829	0.838	0.838	0.841	0.845	0.849	0.844
LoRA (svd)	0.767	0.798	0.818	0.825	0.830	0.830	0.835	0.841	0.841	0.844

Table 8: Per-epoch Quora development accuracy for the four paraphrase runs (single seed). The epoch-9 column corresponds to the “Dev acc” values in Table 5.

C-Qualitative Taxonomy of Sonnet Generation

A qualitative analysis was conducted to identify recurring characteristics in the generated sonnets. The model generally preserves the conditioning lines and adopts vocabulary associated with Shakespearean-style language. Structural cues, including the closing-couplet layout, are also reproduced in several outputs. However, the generated text does not consistently follow the strict 14-line iambic-pentameter structure. Lexical artifacts and semantic drift are also occasionally observed, especially in later parts of the generated poems. These observations complement the quantitative chrF and BLEU scores by highlighting stylistic and structural properties that cannot be fully represented by automatic metrics alone.

Category	Description	Example (real generated output)
Seed fidelity	The three conditioning lines are reproduced verbatim.	“Thine eyes I love, and they, as pitying me, / Knowing thy heart torment me with disdain,”
Archaic diction	Period-appropriate vocabulary and contractions are adopted.	“Whate’er I have gathered. . .”, “Lo, however. . .”
Structural cue	The indented closing-couplet layout is preserved.	“ Lo, however no dirty day glory will blame thee. . .”
Length/meter deviation	Output departs from the strict 14-line iambic-pentameter form.	Generated stanzas vary in line count and rhythm.
Lexical artifacts	Coined or non-words appear.	“desexor”, “priggish pompous verse”
Semantic drift	Thematic coherence degrades over the stanza.	Later lines lose connection to the opening conceit.

Table 9: Taxonomy of generation behaviors observed in full-model sonnets, with verbatim examples.

D. Abbreviations and Terminology

Abbreviation / Term	Definition
GPT-2	Generative Pre-trained Transformer 2; the decoder-only Transformer architecture used as the model backbone.
LoRA	Low-Rank Adaptation; a parameter-efficient method that freezes pretrained weights and learns low-rank updates.
PEFT	Parameter-Efficient Fine-Tuning; a family of methods that adapts pretrained models while updating only a small subset of parameters.
AdamW	Adam optimizer with decoupled weight decay.
SVD	Singular Value Decomposition; used in this study to initialize LoRA factors according to dominant singular directions of the pretrained weight matrix.
Q / K / V	Query, Key, and Value projections in the self-attention mechanism. LoRA adapters are applied to these projections in all 12 Transformer layers.
SST	Stanford Sentiment Treebank; the five-class sentiment classification dataset used in the experiments.
CFIMDB	The binary sentiment classification dataset used in the sentiment experiments.
chrF	Character n-gram F-score; the primary metric used for evaluating sonnet generation.
BLEU	Bilingual Evaluation Understudy; a word n-gram precision-based metric with a brevity penalty, used as a complementary generation metric.

Macro-F1	The arithmetic mean of the F1 scores calculated independently for each class.
Full fine-tuning	A training regime in which all parameters of the pretrained GPT-2 model are updated.
Last-linear	A regime in which the GPT-2 backbone is frozen and only the final task-specific linear layer is trained.
Default initialization	LoRA initialization in which one factor is randomly initialized and the other is initialized to zero, producing an initial zero update.
Weight-dist	LoRA initialization based on the mean and standard deviation of the corresponding pretrained weight matrix.
Dev set	A development dataset used to evaluate model performance during experimentation.
Trainable parameters	Parameters whose values are updated during optimization.
Rank r	The dimensionality of the low-rank LoRA decomposition; $r=8$ is used in this study.
α/alpha	The LoRA scaling parameter; $\alpha=16$ is used in the experiments.